

Stage 1 - Seed input collection

Everything starts here. Accept the raw identifiers that define the target organisation's known digital footprint. These seeds are the entry points for every subsequent discovery step.

Manual entry / CSV upload -> seeds.json -> Stage 2

Accepted input types

Seed normaliser

Accept these types in your intake form/API. Store them all in one `seeds` table with a `type` column.

```
# Seeds JSON schema - what Stage 2 consumes
{
  "org_name": "Example Corp",
  "domains": ["example.com", "example.io"],
  "ip_ranges": ["203.0.113.0/24", "198.51.100.0/24"],
  "asns": ["AS15169"],
  "emails": ["admin@example.com"],
  "github": ["examplecorp"],
  "aws_accounts": ["123456789012"]
}
```

For ASN → IP range expansion, query BGP data:

```
curl "https://stat.ripe.net/data/as-overview/data.json?resource=AS61138" | jq
```

Stage 2 - Passive reconnaissance

Discover as many subdomains, hosts, and related assets as possible without ever touching the target directly. All data comes from third-party sources: DNS records, certificate logs, search engines, and public APIs.

domains from seeds.json -> raw_hosts.txt -> Stage 3

Subfinder

Queries 50+ passive sources (VirusTotal, SecurityTrails, Shodan, etc.) in parallel. Fastest starting point.

```
# Basic run - outputs one host per line
subfinder -d example.com -o subfinder_out.txt

# With API keys configured for more sources
subfinder -d example.com -all -o subfinder_out.txt

# Process a list of root domains
subfinder -dL domains.txt -o subfinder_out.txt -t 20
```

Amass (passive mode)

More thorough than Subfinder; also maps relationships between assets. Use passive-only to stay quiet.

```
# Passive only - no active DNS queries
amass enum -passive -d example.com -o amass_out.txt

# Output graph to Neo4j (great for Stage 8)
amass enum -passive -d example.com \
  -neo4j "bolt://localhost:7687" -o amass_out.txt
```

Certificate transparency (crt.sh)

Every TLS certificate issued is logged publicly. Wildcard certs often reveal internal subdomains that DNS enumeration misses.

```
# Pull all certs for a domain, extract unique names
curl -s "https://crt.sh/?q=example.com&output=json" \
  | jq -r '.[].name_value' \
  | sed 's/\*\.//g' \
  | sort -u \
  >> raw_hosts.txt
```

Shodan API

Shodan has already scanned the internet - you're just querying its database. Finds hosts by org name, ASN, CIDR, or SSL cert fingerprint.

```
# Search by org name - finds all indexed hosts
shodan search 'org:"Example Corp"' \
  --fields ip_str,port,hostnames,vulns \
  --separator , > shodan_out.csv

# Search by CIDR
shodan search net:203.0.113.0/24 \
  --fields ip_str,port,product,version > shodan_cidr.csv

# Python - get all host details for an IP
import shodan
api = shodan.Shodan("YOUR_API_KEY")
host = api.host("203.0.113.5")
```

theHarvester

Collects emails, names, subdomains, IPs from Google, Bing, LinkedIn, Baidu, and more. Great for finding employee identities and shadow IT.

```
# Query all sources, save JSON report
theHarvester -d example.com -b all \
  -f harvest_report
```

```
# Specific sources only (faster)
theHarvester -d example.com \
  -b google,linkedin,bing,censys \
  -l 500 -f harvest_report
```

Stage 3 - DNS resolution & validation

Passive recon gives you thousands of potential hostnames - most are stale or duplicates. This stage resolves them all to confirm which are live, and maps each hostname to its actual IP address(es).

raw_hosts.txt (unvalidated) -> resolved.json (live hosts + IPs) -> Stage 4 & 5

dnsx

The workhorse for bulk DNS resolution. Gets A, AAAA, CNAME, MX, TXT records and outputs structured JSON. Fast and reliable.

```
# Resolve all hosts, get IPs, filter live only
dnsx -l raw_hosts.txt \
  -a -resp -o resolved.txt \
  -json > resolved.json
```

```
# Also grab CNAME (exposes takeover candidates)
dnsx -l raw_hosts.txt \
  -a -cname -resp \
  -json > dns_full.json
```

```
# Wildcard detection - skip wildcard domains
dnsx -l raw_hosts.txt \
  -a -resp -wd example.com \
  -json > resolved_no_wildcard.json
```

The `-cname` flag is critical - dangling CNAMEs pointing to unclaimed services are subdomain takeover candidates, flagged automatically in Stage 7.

MassDNS (for massive lists)

If `raw_hosts.txt` has 1M+ entries, MassDNS resolves them all in minutes using a list of public resolvers in parallel.

```
# Resolve 1M hosts in ~5 minutes
./bin/massdns \
  -r resolvers.txt \           # public resolver list
  -t A raw_hosts.txt \
  -o S \                       # simple output format
  -w massdns_out.txt

# Get resolvers list (trusted public resolvers)
curl -o resolvers.txt \
  https://raw.githubusercontent.com/janmasarik/resolvers/master/resolvers.txt
```

Stage 4 - Port scanning & service discovery

For every live IP discovered in Stage 3, find which ports are open and what services are running on them. This is a three-layer process: fast scan to find open ports → version detection → banner grabbing for full fingerprint.

resolved IPs + hosts -> ports.json (IP:port:service) -> Stage 5, 7

naabu - fast port discovery

Run this first. It's the fastest way to find which ports are open across a large host list. Pipe its output directly into Nmap for deep scanning.

```
# Scan top 1000 ports across all live hosts
naabu -l resolved.txt \
  -top-ports 1000 \
  -o open_ports.txt \
  -json > ports.json

# Full 65535 port range (slower, more thorough)
naabu -l resolved.txt \
  -p - \
  -rate 1000 \
  -o all_ports.txt

# Pipe directly into Nmap for service detection
naabu -l resolved.txt -top-ports 1000 \
  | nmap -sV -iL - -oA nmap_results
```

Nmap - service & version detection

Run Nmap only on the open ports naabu found - never full-range on a large target. Use `-sV` for version detection and `-sC` for default scripts.

```
# Service + version + default NSE scripts
nmap -sV -sC \
  -iL open_ports.txt \
  --open \
  -oA nmap_results \
  -oX nmap_results.xml

# Target specific ports from naabu output
nmap -sV -p 80,443,8080,8443,22,3306 \
  -iL resolved.txt \
  --open -oJ nmap_results.json

# OS detection (requires root)
sudo nmap -O -sV -iL resolved.txt \
  --open -oA nmap_full
```

ZGrab2 - banner grabbing

Where Nmap stops, ZGrab2 starts. It grabs full TLS cert details, HTTP headers, SSH banners, FTP banners - all in structured JSON. Essential for enriching your asset graph.

```
# Grab HTTP + HTTPS details from all web hosts
cat web_hosts.txt | zgrab2 multiple \
  --senders=200 \
  -m zgrab2_modules.ini \
  -o zgrab2_out.json
```

```
# Example module config (zgrab2_modules.ini)
[http]
port = 80
[tls]
port = 443
[ssh]
port = 22
```

```
# TLS only - extract cert details
cat https_hosts.txt | zgrab2 tls \
  --port 443 -o tls_out.json
```

Stage 5 - HTTP probing & web fingerprinting

From all the open ports, identify which are web services, then crawl and fingerprint each one. This stage maps your entire web attack surface: endpoints, technologies, response codes, login pages, and historical URLs.

open_ports.json -> web_assets.json -> Stage 6, 7

httpx - probe all web hosts

This is your most important Stage 5 tool. In one command it tells you: is this a live web server? What does it respond with? What tech is running? What's the page title? Run this first.

```
# The full enrichment run - mostuseful command
httpx -l resolved.txt \
  -title \           # page title
  -tech-detect \     # tech stack (Wappalyzer rules)
  -status-code \     # HTTP status
  -content-length \
  -web-server \      # Server header
  -follow-redirects \
  -screenshot \      # visual screenshot of page
  -o httpx_out.json \
  -json

# Filter to only alive web apps
httpx -l resolved.txt \
  -mc 200,301,302,403 \
  -o alive_web.txt
```

The `-screenshot` flag is a hidden gem - it captures a screenshot of every live web app so you can visually review the attack surface without visiting each URL.

Katana - endpoint crawler

Crawls web apps to find hidden endpoints, forms, and JavaScript-rendered paths. Use depth 3 as a good balance of coverage vs speed.

```
# Standard crawl with JS rendering enabled
katana -u https://example.com \
  -d 3 \                # crawl depth
  -jc \                 # JS form completion
  -kf all \             # keep all known fields
  -o katana_out.txt

# Crawl all alive web apps from httpx output
cat alive_web.txt | katana \
  -d 2 -jc \
  -o endpoints.txt \
  -json > endpoints.json
```

waybackurls - historical endpoints

Queries the Wayback Machine and Common Crawl for historical URLs. Finds old API endpoints, removed admin panels, and forgotten paths that may still be live.

```
# Get all historical URLs for a domain
waybackurls example.com | tee wayback_out.txt

# Combine with gau (GetAllURLs) for more coverage
go install github.com/lc/gau/v2/cmd/gau@latest gau --threads 5 example.com | tee
gau_out.txt

# Find old API endpoints and params cat wayback_out.txt | grep -E
"\.json|api/|/v[0-9]" \ > old_api_endpoints.txt
```

Stage 6 - Cloud & code scanning

Modern attack surface extends beyond web servers into cloud storage, misconfigured accounts, and leaked credentials in code repositories. This stage finds all of that - often the highest-impact discoveries in the entire pipeline.

org name, GitHub orgs, cloud IDs -> cloud_findings.json, secrets.json -> Stage 9 (direct to critical)

TruffleHog - secrets in git repos

Scans all git history (not just current code) for API keys, passwords, and credentials. Finds secrets developers committed and then deleted - they're still in git history.

```
# Scan an entire GitHub org for secrets
trufflehog github \
  --org=examplecorp \
  --json > trufflehog_out.json

# Scan a specific repo including full history
trufflehog git \
  https://github.com/examplecorp/backend \
  --json > secrets.json

# Scan local filesystem
trufflehog filesystem \
  /path/to/cloned/repos \
  --json > local_secrets.json
```

S3Scanner - open cloud buckets

Checks S3, GCS, Azure Blob, and DigitalOcean Spaces for public access. Generate bucket name guesses from the org name and known domain patterns.

```
# Generate bucket name candidates
echo "example-corp example-backup example-prod example-dev example-assets example-logs" > buckets.txt

# Scan across all major cloud providers
s3scanner --bucket-file buckets.txt \ --provider aws,gcp,azure,do \ --threads 20 \
--out-file bucket_results.json
```

Prowler - cloud misconfiguration

Runs 500+ checks against AWS, GCP, and Azure for security misconfigurations. Covers IAM, S3, EC2, RDS, Lambda, networking. Requires cloud credentials.

```
# Full AWS audit - outputs JSON + HTML report
prowler aws \
  -M json,html \
  -o prowler_output/ \
  --log-level ERROR

# GCP and Azure
prowler gcp --project-ids your-project-id
prowler azure --sp-env-auth

# Focus on critical checks only
prowler aws -S \      # severity: CRITICAL,HIGH
  -M json -o critical_findings/
```

Stage 7 - Vulnerability scanning

With a fully mapped attack surface, now you systematically probe each asset for known vulnerabilities. Nuclei is the backbone - it uses a community-maintained library of 9000+ detection templates covering CVEs, misconfigurations, exposed panels, and takeover candidates.

web_assets.json, ports.json -> vulns.json (findings with severity) -> Stage 9

Team associated with Vulnerability scanning and asset monitoring can look into it

Stage 8 - Asset storage & graph database

All tool outputs converge here into a unified, queryable data store. Use PostgreSQL for structured asset data and Neo4j for the relationship graph - attack paths, ownership chains, and blast-radius analysis all require the graph layer.

all JSON outputs from stages 2-7 -> PostgreSQL + Neo4j -> Stage 9

PostgreSQL - asset inventory schema

Store all discovered assets here. Every asset has a type, source (which tool found it), discovery timestamp, and current state. Use JSONB for flexible tool-specific metadata.

```
-- Core assets table
CREATE TABLE assets (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type        TEXT NOT NULL,      -- domain|ip|port|endpoint|cert
  value       TEXT NOT NULL,
  org_id      UUID REFERENCES organisations(id),
  first_seen  TIMESTAMPTZ DEFAULT NOW(),
  last_seen   TIMESTAMPTZ DEFAULT NOW(),
  is_active   BOOLEAN DEFAULT TRUE,
  source      TEXT[],             -- ['subfinder','crt.sh']
  metadata    JSONB               -- tool-specific extra data
);

-- Vulnerability findings
CREATE TABLE findings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  asset_id    UUID REFERENCES assets(id),
  tool        TEXT,               -- 'nuclei','nikto','shodan'
  template_id TEXT,               -- e.g. CVE-2021-44228
  severity    TEXT,               -- critical|high|medium|low|info
  title       TEXT,
  description TEXT,
  evidence    JSONB,
  cvss_score  NUMERIC(4,1),
  cisa_kev    BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  resolved_at TIMESTAMPTZ
);
```

Neo4j - attack surface graph

The graph layer is what makes ASM powerful. Relationships between assets reveal attack paths that a flat database cannot express - e.g. "find all paths from internet to internal DB through exposed services."

```
// Node labels: Organisation, Domain, IP, Port, Endpoint, Cert, Finding

// Create nodes
CREATE (:Domain {name: "example.com", active: true})
CREATE (:IP {address: "203.0.113.5", asn: "AS15169"})
CREATE (:Port {number: 443, protocol: "tcp", service: "https"})
CREATE (:Endpoint {url: "https://example.com/admin"})

// Relationships (the attack surface graph)
// Domain resolves to IP
MATCH (d:Domain {name:"api.example.com"}),(i:IP {address:"203.0.113.5"})
CREATE (d)-[:RESOLVES_T0]->(i)

// IP exposes port
MATCH (i:IP),(p:Port {number:443})
CREATE (i)-[:EXPOSES]->(p)

// Port hosts endpoint
MATCH (p:Port),(e:Endpoint)
CREATE (p)-[:HOSTS]->(e)

// Finding affects asset
MATCH (e:Endpoint),(f:Finding {severity:"critical"})
CREATE (f)-[:AFFECTS]->(e)

// Attack path query: internet-exposed endpoints
// with critical findings, tracing back to org
MATCH path = (o:Organisation)-[:OWNS*]->(a)-[:RESOLVES_T0|EXPOSES|HOSTS*]->
(e:Endpoint)
<-[:AFFECTS]-(f:Finding {severity:"critical"})
RETURN path ORDER BY f.cvss_score DESC LIMIT 25
```

Stage 9 - Prioritization, scoring & alerting

With thousands of assets and findings, this final stage produces an actionable risk-ranked queue. It applies the scoring formula, integrates external threat intel (CISA KEV), deduplicates findings, and fires alerts for new critical discoveries.

findings + asset graph -> ranked queue + alerts -> Security team

Team associated with Risk Scoring can look into it